SDA WEEK 1
SDLC

Syed Ahmed Khan
syed.ahmed@nu.edu.pk

SDLC

Software Development Life Cycle (SDLC) is a structured process used to design, develop, test, deploy, and maintain software systems in a systematic and disciplined way.

SDLC

What happens if software is developed without a proper process?

Typical issues:

- Missed requirements
- Cost overrun
- Late delivery
- Poor quality

SDLC

SDLC provides:

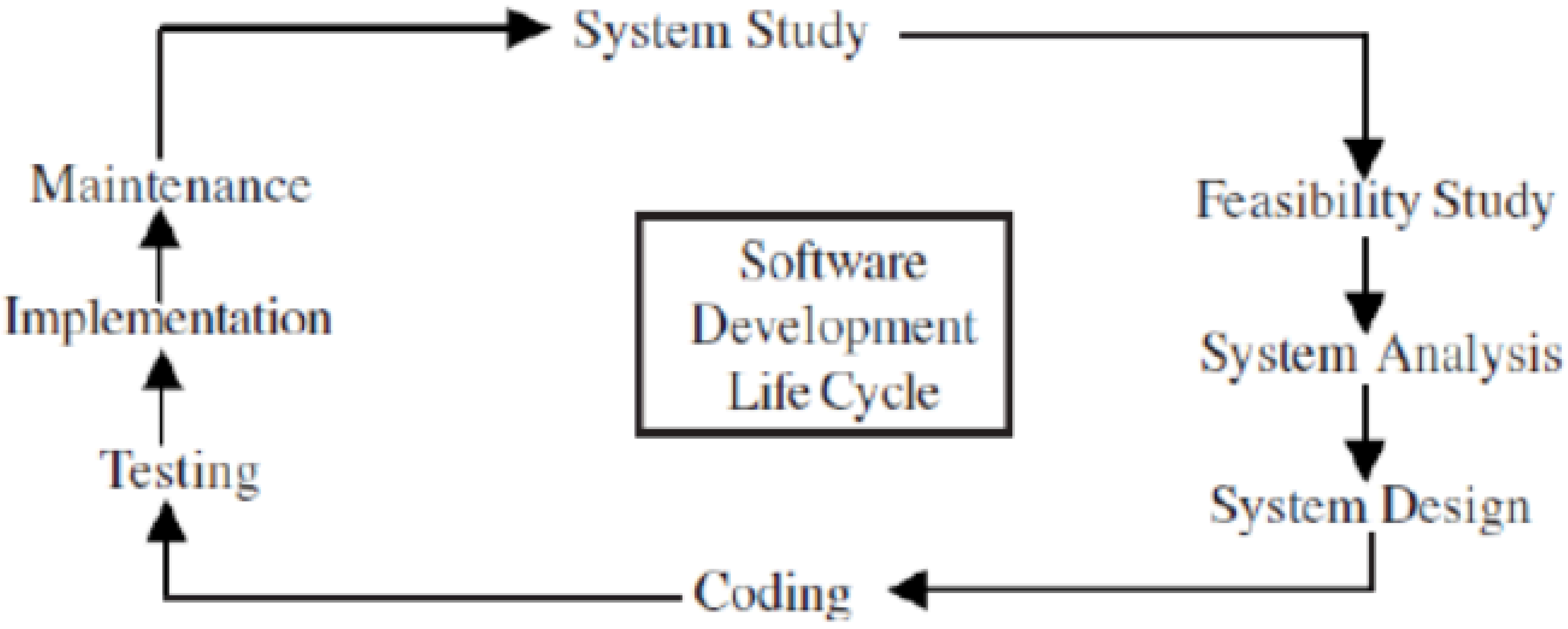
- Structure
- Control
- Predictability
- Quality

SDLC

SDLC divides software development into logical phases, where each phase has:

- specific objectives
- defined outputs
- review checkpoints

SDLC



System Study
Feasibility Study
System Analysis
System Design
Coding
Testing
Implementation
Maintenance
Software
Development
Life Cycle

SDLC

Standard Phases of SDLC

1. System Study
2. Feasibility Study
3. System Analysis
4. System Design
5. Coding (Implementation)
6. Testing
7. Deployment (Implementation)
8. Maintenance

1.SYSTEM STUDY

System Study is the initial investigation of the existing system to understand the problem, scope, and current limitations.

Objectives

- Understand current system
- Identify problems
- Define project scope

Activities

- Initial survey
- Background analysis
- Problem identification

1.SYSTEM STUDY

Example

Existing manual library system:

- Records on paper
- Slow searching
- High error rate

Output

- System study report
- Problem statement
- Project proposal

2.FEASIBILITY STUDY

Feasibility study evaluates whether the proposed system is viable and worth developing.

Types of Feasibility

1. Technical feasibility
2. Economic feasibility
3. Operational feasibility
4. Schedule feasibility

2.FEASIBILITY STUDY

Example

ATM system:

- Technical: Can bank support servers?
- Economic: Is cost justified?
- Operational: Will staff use it?
- Schedule: Can it be built in time?

Output

- Feasibility report
- Go / No-Go decision

3.SYSTEM ANALYSIS

System Analysis is the detailed study of requirements and user needs to define what the new system must do.

Key Focus

- WHAT the system should do
- NOT how to implement it

Activities

- Requirement gathering
- Interviews
- Questionnaires
- Observation

3.SYSTEM ANALYSIS

Tools Used

- Data Flow Diagrams (DFDs)
- Use Case Diagrams
- Data Dictionary

Example

Hospital system:

- Patient registration
- Appointment scheduling
- Billing

Output

- Software Requirement Specification (SRS)

4.SYSTEM DESIGN

System Design converts requirements into a blueprint that can be implemented.

Example

Library system design:

- Tables: Book, Student, Issue
- Classes: Book, Member

Output

- System Design Specification (SDS)
- UML diagrams

5.IMPLEMENTATION

Coding is the phase where design is translated into executable programs using a programming language.

Key Points

- Follow coding standards
- Write modular code
- Use version control

5.IMPLEMENTATION

Example

Implementing:

- Book class
- Issue class

Output

- Source code
- Executable software

6.TESTING

Testing is the process of finding and fixing defects to ensure the software meets requirements.

Types of Testing

1. Unit Testing
2. Integration Testing
3. System Testing
4. User Acceptance Testing (UAT)

6.TESTING

Example

ATM testing:

- Withdraw valid amount
- Insufficient balance
- Wrong PIN

Output

- Test reports
- Bug reports

7.DEPLOYMENT

Deployment is the process of delivering the software to users and making it operational.

Deployment Strategies

1. Direct Cutover
2. Parallel Run
3. Pilot Run
4. Phased Implementation

8.MAINTENANCE

Maintenance is the process of modifying software after deployment to correct faults, improve performance, or adapt to changes.

Types of Maintenance

1. Corrective
2. Adaptive
3. Perfective
4. Preventive

SDLC SUMMARY

SDLC provides a disciplined framework that ensures software is developed systematically, efficiently, and with high quality.

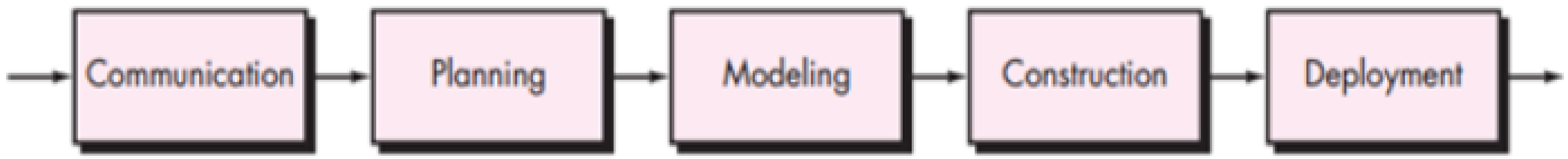
SDLC MODELS (BRIEF OVERVIEW)

Models define how phases are arranged

- Waterfall
- Incremental
- Spiral
- Agile

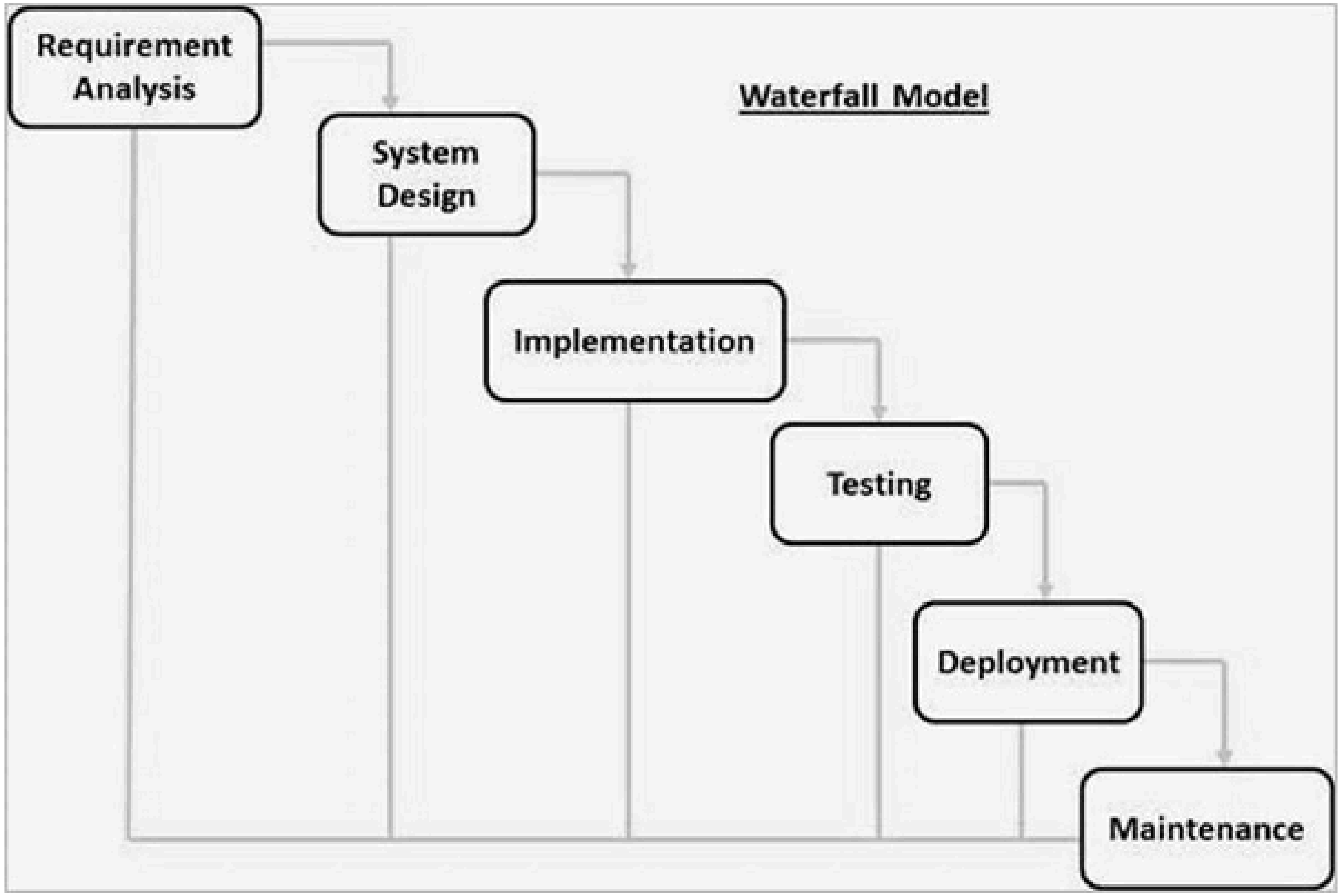
WATERFALL

The Waterfall Model is a linear and sequential SDLC model where each phase must be completed before the next phase begins.



(a) Linear process flow

WATERFALL


Requirement Analysis
Waterfall Model
System Design
Implementation
Testing
Deployment
Maintenance

WATERFALL

Pros

- Simple and easy to understand.
- Works well for smaller projects where the requirements are well-understood.

Cons

- No working product
- Inflexible

WATERFALL

Key Characteristics of Waterfall

- Sequential execution
- Heavy documentation
- Minimal customer involvement after requirements
- Rigid structure

1. Requirements must be specified.
2. Four main tasks must be completed in sequence: *Analysis*, *design*, *code*, and *test*, followed by integration of the system.
3. Output of one stage feeds into the next stage in sequence

WATERFALL

Advantages of Waterfall Model

- Simple and easy to understand
- Well-defined phases
- Easy to manage and control
- Good for small, stable projects

WATERFALL

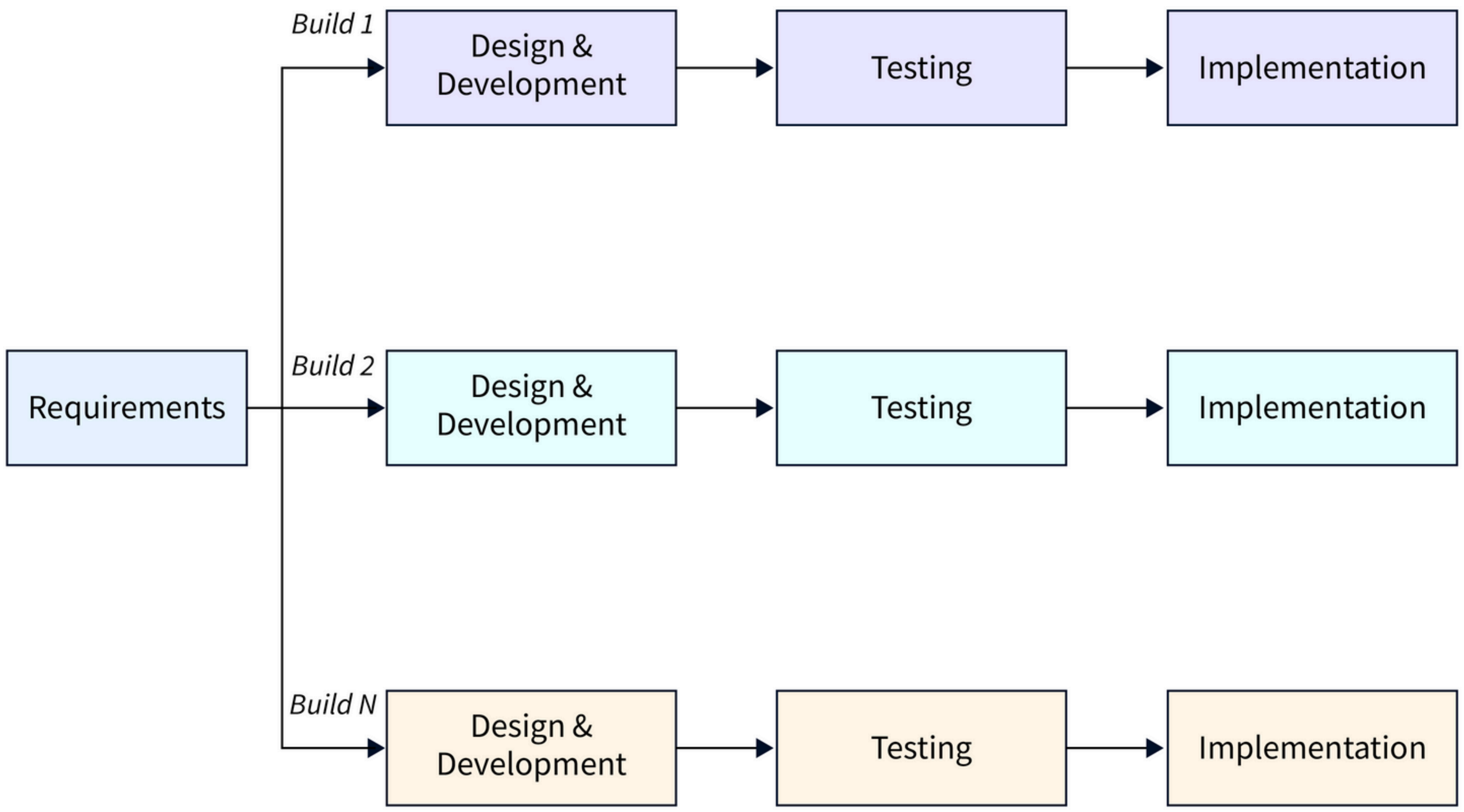
Disadvantages of Waterfall Model

- Very difficult to handle changes
- Working software is available very late
- High risk if requirements are wrong

INCREMENTAL MODEL

- Each “major requirement/item” is further developed separately through the same sequence of :requirement, design, code, and unit test.
- As the developed pieces are completed, they are continuously merged and integrated into a common bucket for integrated system test

INCREMENTAL MODEL



INCREMENTAL MODEL

INCREMENTAL MODEL Pros

- Generates working software quickly
- More flexible
- Less costly
- Easier to test and debug.
- Customer can respond to each built

INCREMENTAL MODEL

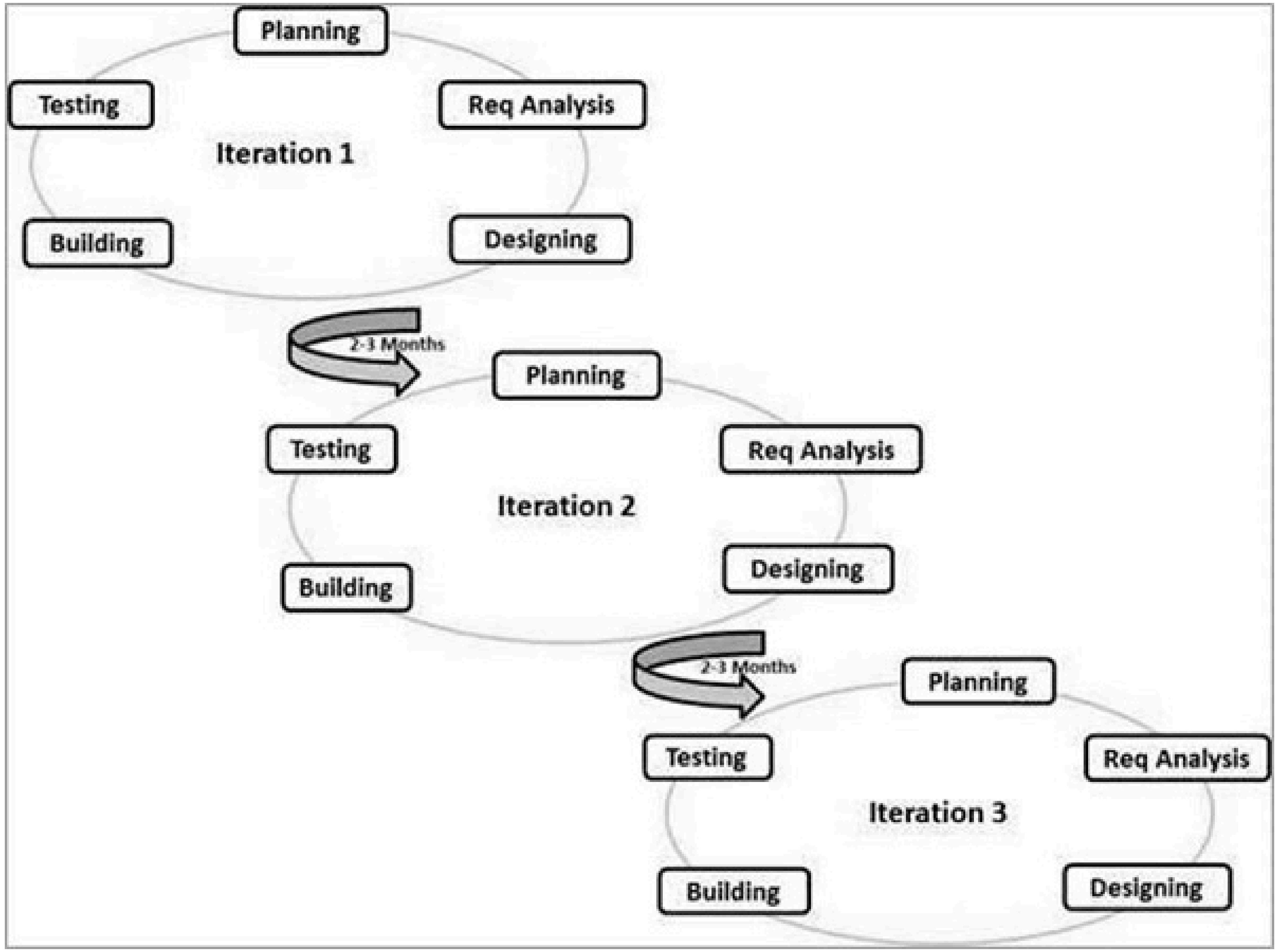
INCREMENTAL MODEL Cons

- Complexity in Integration
- Continuous Maintenance Overhead
- Needs good planning and design
- Needs a clear and complete definition of the whole system

AGILE MODEL

- Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product.
- Agile Methods break the product into small incremental builds. These builds are provided in iterations.
- Each iteration typically lasts from about one to three weeks.

AGILE MODEL



Planning
Testing
Req Analysis
Iteration 1
Building
Designing
2-3 Months
Planning
Testing
Req Analysis
Iteration 2
Building
Designing
2-3 Months
Planning
Testing
Req Analysis
Iteration 3
Building
Designing

AGILE MODEL

Agile follows the Agile **Manifesto**, which values:

- Individuals and interactions over processes
- Working software over documentation
- Customer collaboration over contracts
- Responding to change over following a plan

AGILE MODEL

Instead of one big cycle, Agile works in iterations (sprints).
Each sprint includes:

- Planning
- Design
- Coding
- Testing
- Review

Each sprint delivers a working product increment

AGILE MODEL

Example

Mobile Food Delivery App

- Requirements change frequently
- Customer feedback is crucial
- Features added gradually

AGILE MODEL

Sprint 1:

- User login
- Restaurant listing

Sprint 2:

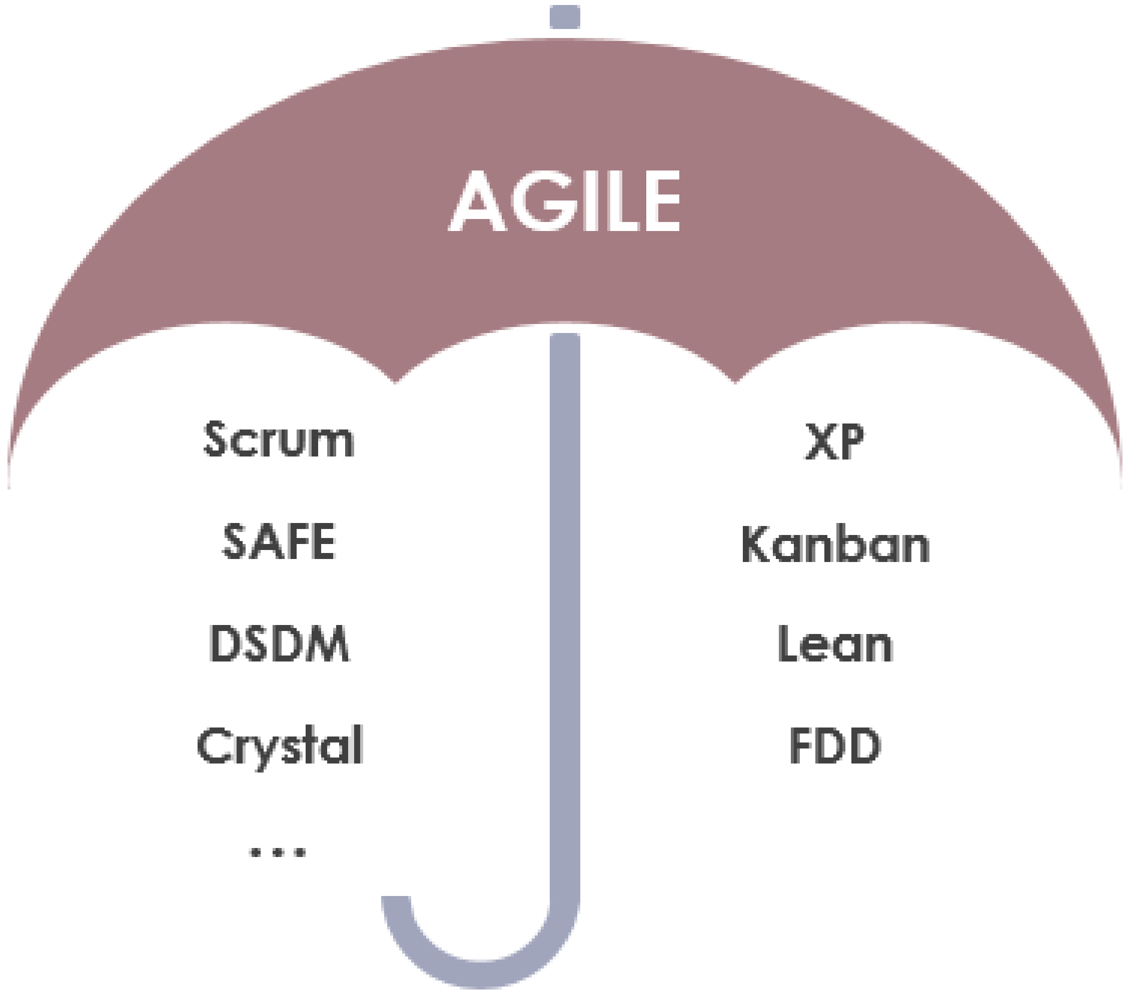
- Order placement
- Payment integration

Sprint 3:

- Tracking
- Reviews

AGILE MODEL

Agile is an umbrella term for a set of methods and practices based on the values and principles expressed in the Agile Manifesto that is a way of thinking that enables teams and businesses to innovate, quickly respond to changing demand, while mitigating risk.



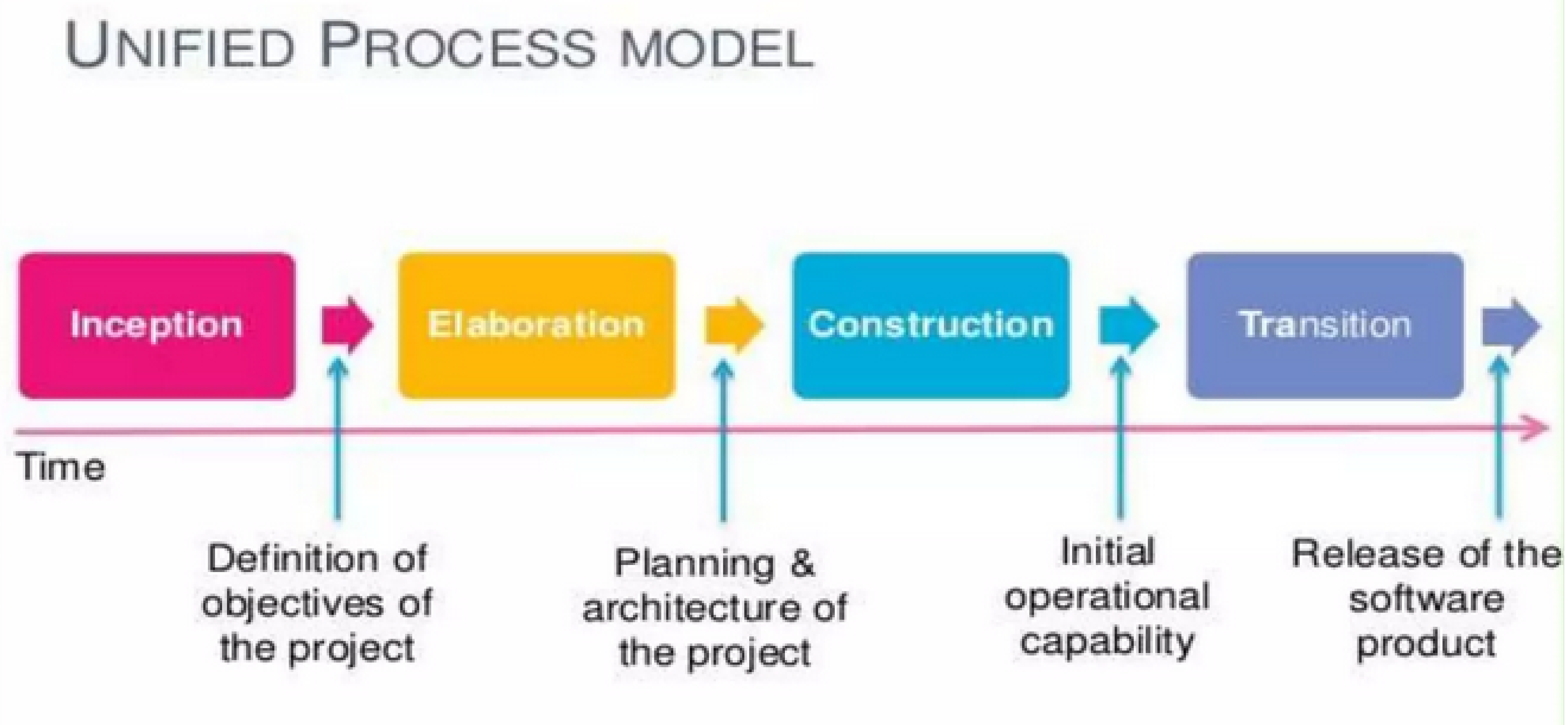
UNIFIED PROCESS MODEL (UP)

The Unified Process is an iterative and incremental software development framework that emphasizes risk reduction, architecture-centric development, and use-case driven design.

UNIFIED PROCESS KEY CHARACTERISTICS

- Iterative & Incremental
  - Iterative: Refine through cycles
  - Incremental: Add features gradually
- Object-Oriented
  - Utilizes object oriented technologies
- Use-Case Driven
  - Requirements captured as use cases
- Architecture-Centric
  - Focus on architectural baseline early
  - "Walking skeleton" approach
- Risk Diven
  - Highest risks addressed earliest

FOUR PHASES



UNIFIED PROCESS MODEL
Inception
Elaboration
Construction
Transition
Time
Definition of objectives of the project
Planning & architecture of the project
Initial operational capability
Release of the software product

INCEPTION PHASE

The inception phase has the following objectives:

- Gathering and analyzing the requirements.
- Planning and preparing a business case and evaluating alternatives for risk management, scheduling resources etc.
- Estimating the overall cost and schedule for the project.
- Studying the feasibility and calculating profitability of the project.

ELABORATION PHASE

The elaboration phase has the following objectives:

- Establishing architectural foundations.
- Design of use case model.
- Elaborating the process, infrastructure & development environment.
- Selecting components.
- Demonstrating that architecture supports the vision at reasonable cost & within specified time.

CONSTRUCTION PHASE

The construction phase has the following objectives:

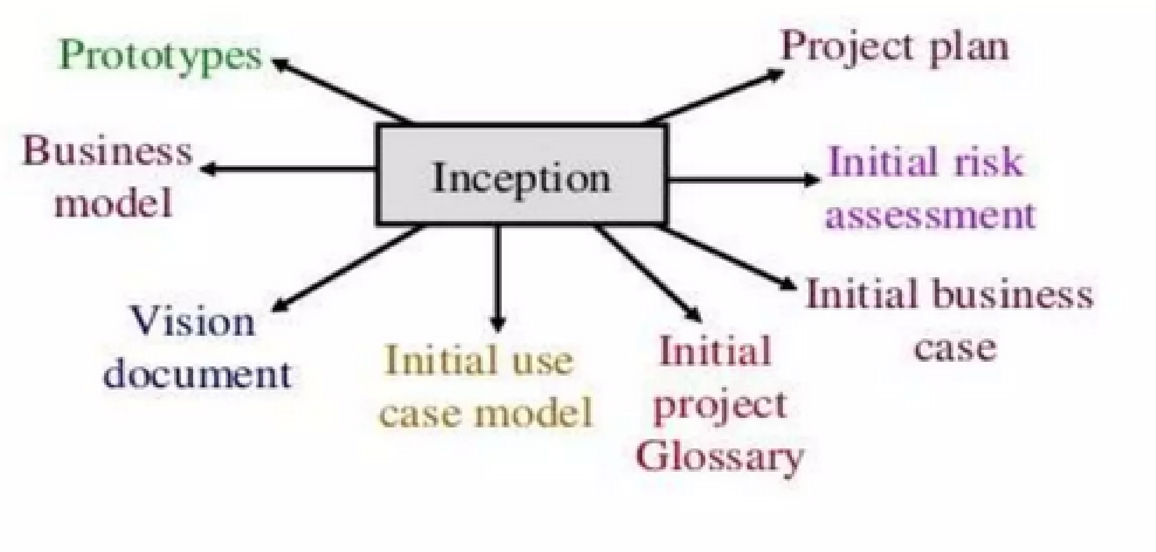
- Implementing the project.
- Minimizing development cost.
- Management and optimizing resources.
- Testing the product.
- Assessing the product releases against acceptance criteria.

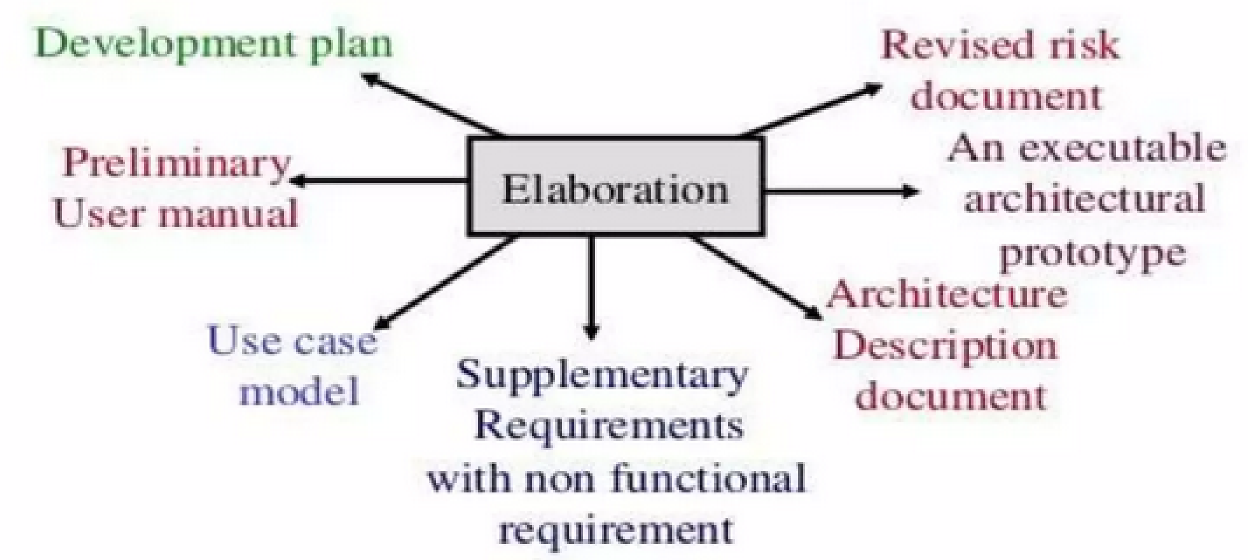
TRANSITION PHASE

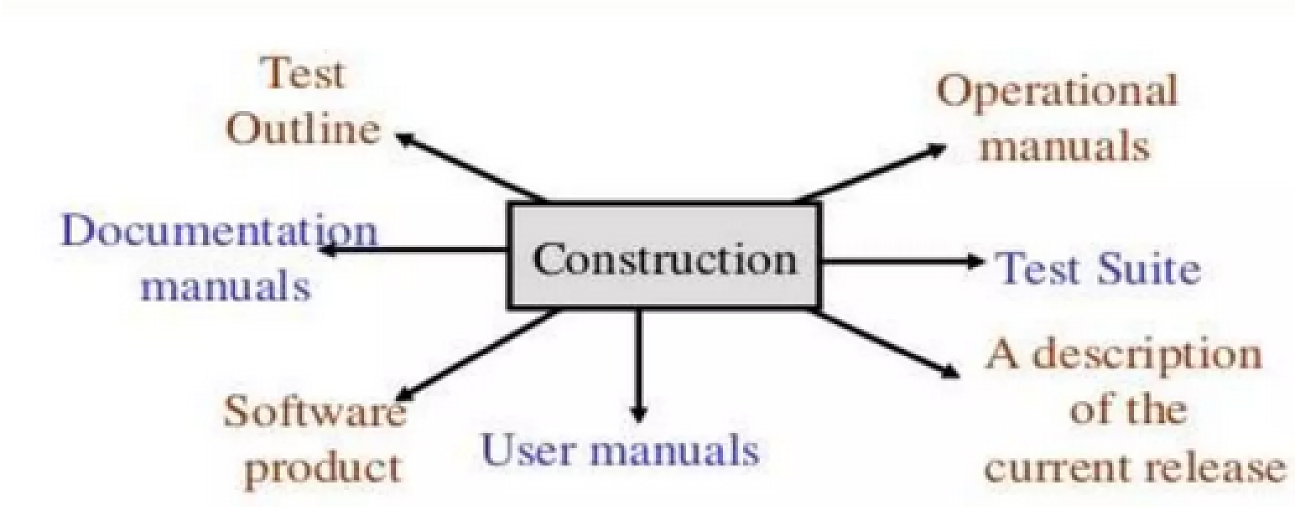
The transition phase has the following objectives:

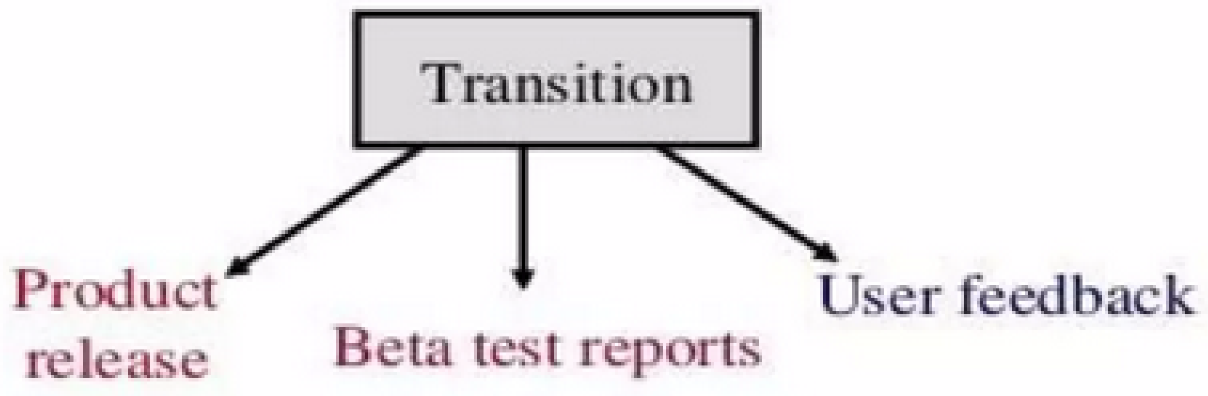
- Starting of beta testing.
- Analysis of user's views.
- Training of users.
- Tuning activities including bug fixing and enhancements for performance & usability.
- Assessing the customer satisfaction.

OUTCOMES FOR EACH PHASE





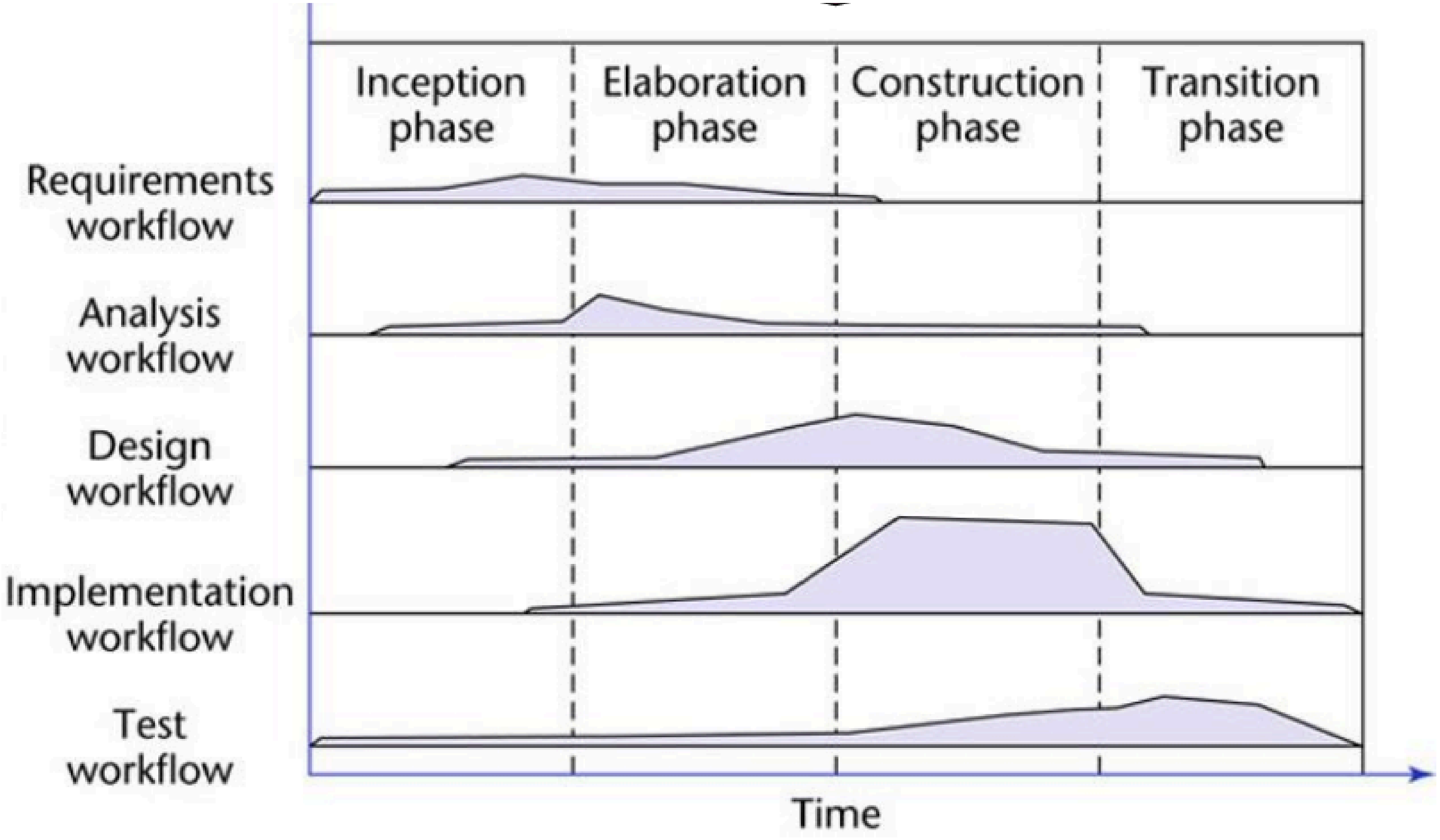




UNIFIED PROCESS KEY CHARACTERISTICS

- The inception phase has the following objectives:
- Gathering and analyzing the requirements.
- Planning and preparing a business case and evaluating alternatives for risk management, scheduling resources etc.
- Estimating the overall cost and schedule for the project.
- Studying the feasibility and calculating profitability of the project.

UNIFIED PROCESS WORKFLOWS



DELIVERABLES BY PHASES

Inception Phase	Elaboration Phase	Construction Phase	Transition Phase
• The initial version of the domain model • The initial version of the business model • The initial version of the requirements artifacts • A preliminary version of the analysis artifacts • A preliminary version of the architecture • The initial list of risks • The initial ordering of the use cases • The plan for the elaboration phase • The initial version of the business case	• The completed domain model • The completed business model • The completed requirements artifacts • The completed analysis artifacts • An updated version of the architecture • An updated list of risks • The project management plan (for the rest of the project) • The completed business case	• The initial user manual and other manuals, as appropriate • All the artifacts (beta release versions) • The completed architecture • The updated risk list • The project management plan (for the remainder of the project) • If necessary, the updated business case	• All the artifacts (final versions) • The completed manuals

THANK YOU